

SISTEMA INTEGRADO DE ATENDIMENTO E EXECUÇÃO DE SERVIÇOS

Tech Challenge — Fase 3

Segurança, observabilidade e operação em nuvem

Danilo Canato — RM 372411

Guilherme Fumagali Marques — RM 372421

PosTech FIAP — Arquitetura de Software

Setembro de 2026

Referências de Entrega

Esta seção reúne os links dos quatro repositórios, do vídeo de demonstração e da documentação arquitetural. O usuário soat-architecture foi adicionado como colaborador em todos os repositórios. A aplicação oficina-api permanece no repositório utilizado nas fases anteriores, tech-challenge-1, de modo a preservar o histórico.

Repositório	Conteúdo	Link
tech-challenge-1	Aplicação oficina-api (Spring Boot)	github.com/Guilherme-Fumagali/tech-challenge-1
oficina-auth-lambda	Function de autenticação por CPF	github.com/Guilherme-Fumagali/oficina-auth-lambda
oficina-infra-k8s	Rede, EKS, ECR, API Gateway, New Relic	github.com/Guilherme-Fumagali/oficina-infra-k8s
oficina-infra-db	RDS PostgreSQL gerenciado	github.com/Guilherme-Fumagali/oficina-infra-db

Artefato	Link
Vídeo de demonstração	https://youtu.be/hT5m2dWdpwI
Swagger / OpenAPI	https://redocly.github.io/redoc/?url=https://raw.githubusercontent.com/Guilherme-Fumagali/tech-challenge-1/develop/docs/openapi.json
RFCs, ADRs, diagramas e débitos técnicos	github.com/Guilherme-Fumagali/tech-challenge-1/tree/main/docs/tech-challenge-3

1. Introdução e Contexto do Problema

A oficina mecânica atendida por este sistema expandiu sua operação para múltiplas unidades, e a base de clientes está em crescimento. Com isso, as necessidades deixaram de se limitar à organização interna e passaram a envolver aspectos operacionais: controle de acesso, visibilidade do comportamento do sistema em tempo real e capacidade de absorver picos de uso sem intervenção manual.

Os requisitos desta fase não acrescentam funcionalidades de negócio. Eles tratam de capacidade operacional: autenticação na borda, segregação de responsabilidades entre repositórios, infraestrutura de nuvem declarada em código e observabilidade com uma plataforma de mercado.

1.1 Situação ao final da Fase 2

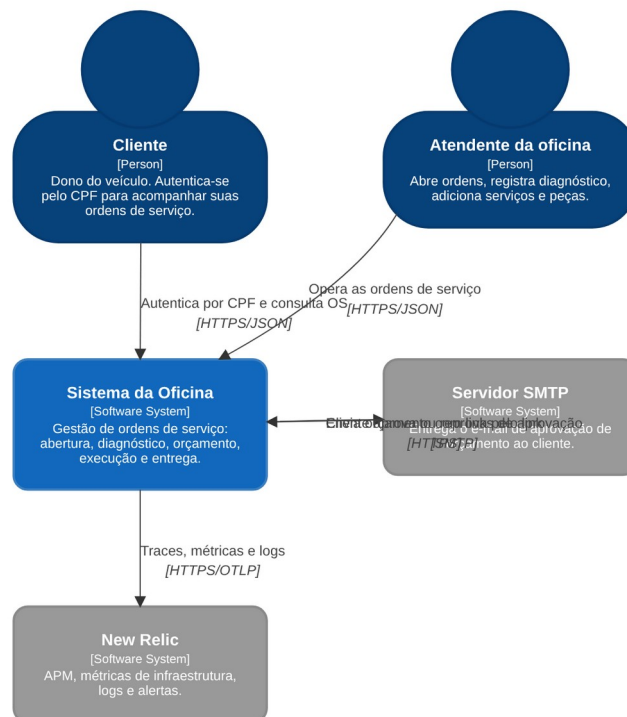
Parte da infraestrutura exigida já havia sido implementada na Fase 2. O levantamento realizado antes do início da implementação indicou que três dos cinco requisitos de infraestrutura estavam atendidos:

Requisito da Fase 3	Situação ao iniciar
Cluster Kubernetes com escalabilidade	EKS com HPA por CPU e memória, já provisionados
Banco de dados gerenciado	RDS PostgreSQL 16, sem acesso público
Terraform para provisionamento	VPC, EKS, RDS e backend de state em S3 e DynamoDB
API Gateway	Inexistente; a proteção era feita apenas pelo Spring Security
Function serverless para autenticação	Inexistente; a própria aplicação emitia o token

A implementação concentrou-se, portanto, em três frentes: a transferência da autenticação para fora da aplicação, a divisão do repositório único em quatro repositórios com pipelines próprios e a instrumentação do sistema para observabilidade.

Figura 1 — Diagrama de contexto (C4, nível 1)

Sistema de Ordens de Serviço da Oficina — Contexto



Fonte: elaborado pelos autores.

2. Evolução da Aplicação

2.1 Emissão de tokens transferida para a função serverless

Até a Fase 2, a autenticação era realizada pela própria aplicação: um AuthController expunha as operações de login, refresh e logout, o JwtService assinava os tokens e um único usuário administrador era mantido em memória.

Na Fase 3, a emissão do token passou a ser responsabilidade de uma função Lambda, e a aplicação ficou restrita à validação. Foram removidos o AuthController, o UserDetailsConfig, o RefreshTokenService e os três DTOs associados. O JwtService manteve apenas a verificação de assinatura e a leitura de claims.

A principal motivação é concentrar a emissão em um único componente, o que simplifica a rotação da chave, padroniza as claims e centraliza a auditoria. A alternativa de manter dois emissores, um para clientes e outro para administradores, foi descartada por criar dois domínios de confiança sobre a mesma API.

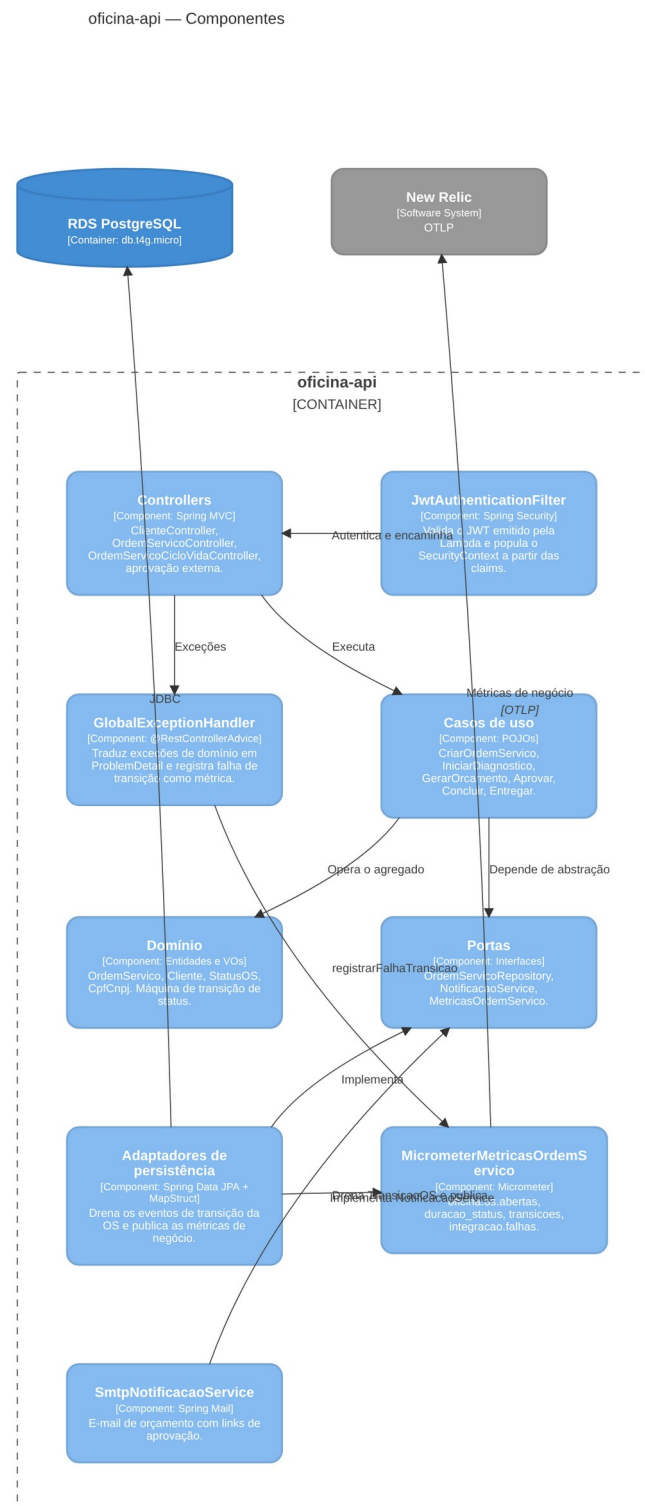
2.2 Métricas de negócio emitidas a partir do domínio

Os três painéis exigidos pelo enunciado (volume diário de ordens, tempo médio por status e erros de integração) não são fornecidos pelo agente de APM, que coleta latência e erros HTTP, mas não possui informação sobre o ciclo de vida de uma ordem de serviço.

Uma alternativa seria injetar o MeterRegistry em cada caso de uso, o que exigiria alterar nove construtores. Optou-se por fazer o agregado OrdemServico registrar eventos de transição em um método único, que também valida a mudança de status e calcula o tempo de permanência no estado anterior. O adaptador de persistência consome esses eventos ao salvar a ordem, ponto pelo qual todas as transições passam.

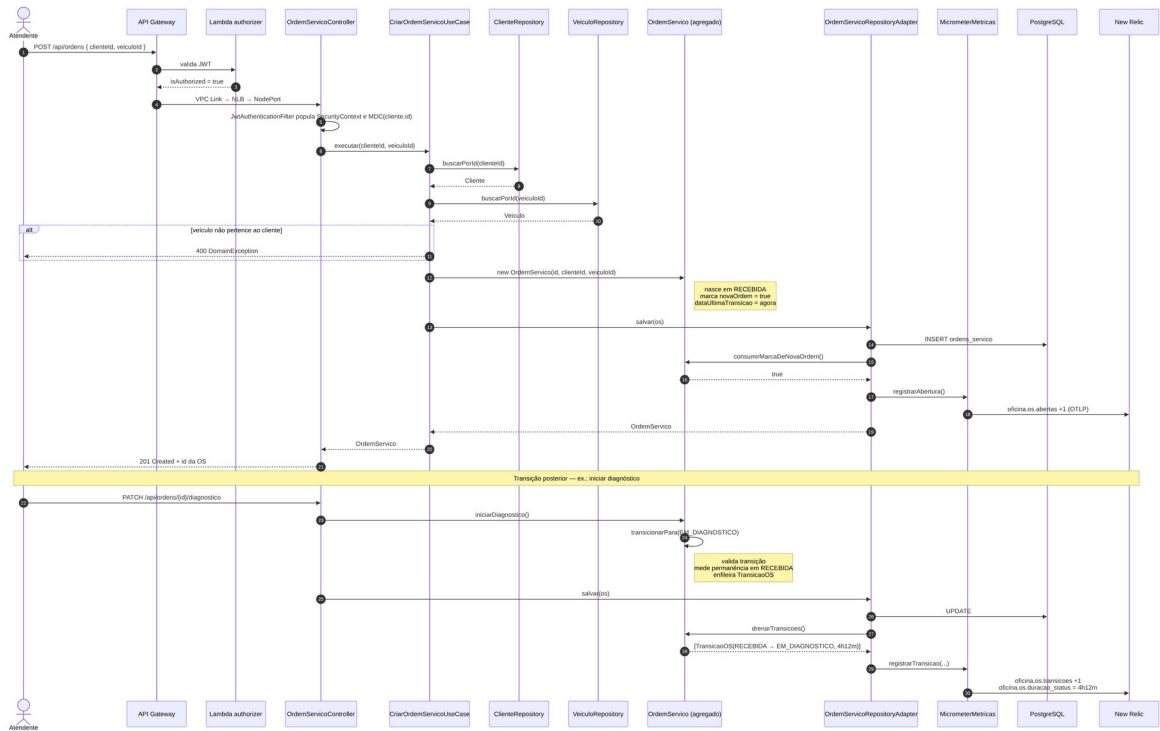
Dessa forma, a métrica representa o evento de negócio, e não a requisição HTTP, e permanece correta caso a ordem passe a ser aberta por outro canal. O domínio também não depende do Micrometer.

Figura 2 — Componentes da aplicação oficina-api (C4, nível 3)



Fonte: elaborado pelos autores.

Figura 3 — Sequência de abertura de ordem de serviço



Fonte: elaborado pelos autores.

2.3 Ajustes no modelo relacional

Foram criadas duas migrations, ambas motivadas por requisitos da fase:

Migration	Mudança	Motivo
V6	clientes.status e constraint	A Lambda consulta a existência e o status do cliente
V6	idx_clientes_cpf_status	Consulta executada a cada autenticação
V6	idx_os_status_abertura	Listagem por prioridade e agregações dos dashboards
V7	ordens_servico.data_ultima_transicao	Necessária para medir a permanência por status

O valor padrão 'ATIVO' na coluna de status mantém a autenticação dos registros existentes após o deploy. O preenchimento da V7 utiliza COALESCE sobre as datas já registradas, que é a melhor aproximação disponível para as ordens antigas.

2.4 Débitos técnicos da Fase 2

A Fase 2 foi encerrada com um débito aberto, o DT-01: o e-mail de aprovação de orçamento apresenta apenas o valor total, sem detalhar os serviços e as peças que o compõem.

O débito permanece aberto, pois o escopo da Fase 3 é infraestrutura, segurança e observabilidade, e nenhum requisito envolve o conteúdo da notificação. A correção está prevista para quando a notificação for migrada para SNS/SES.

3. Autenticação e API Gateway

3.1 Fluxo de autenticação

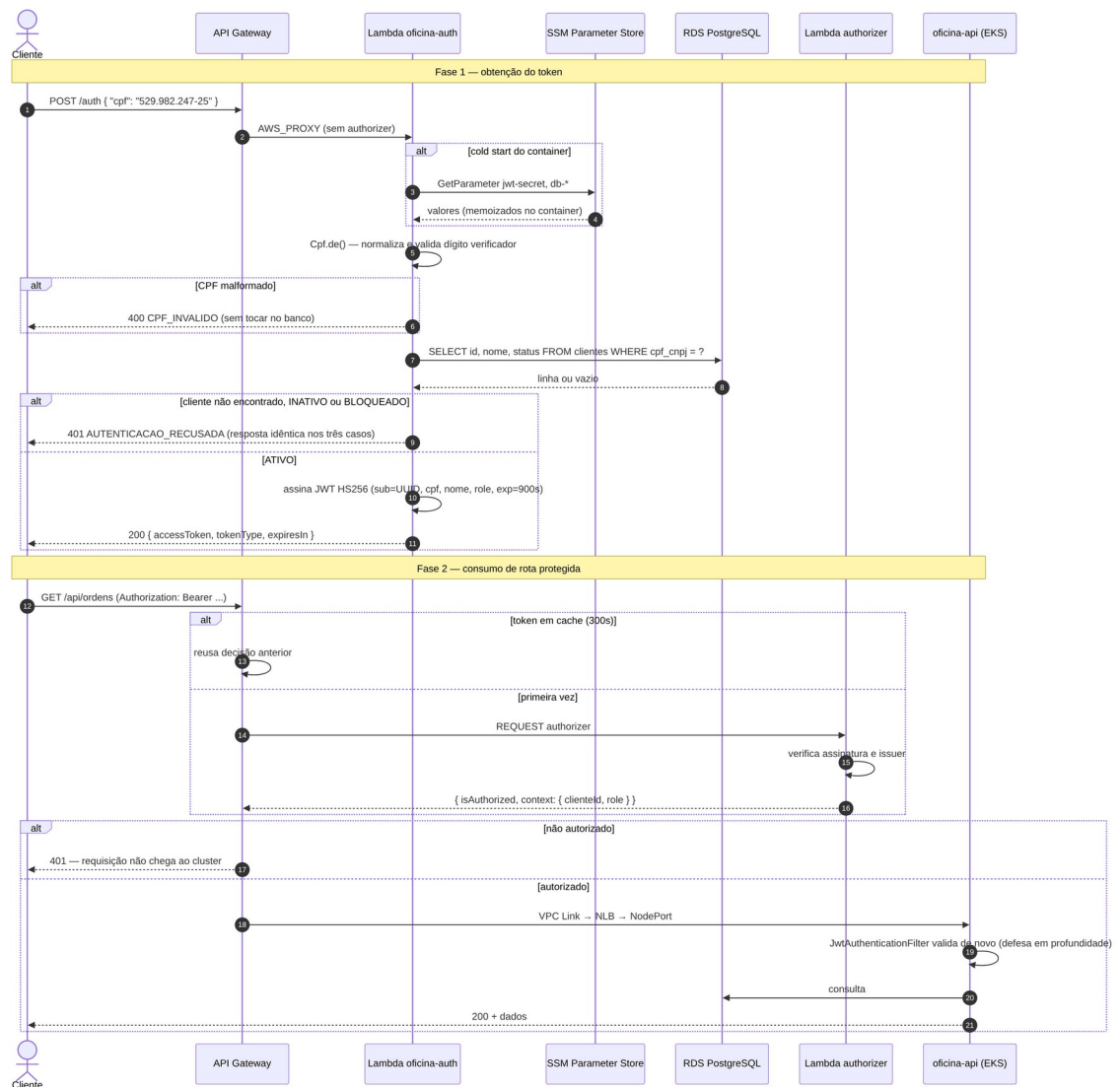
O cliente envia o CPF ao API Gateway, que encaminha a requisição para a função Lambda. A função normaliza o CPF e valida o dígito verificador antes de consultar o banco de dados; em seguida, verifica a existência e o status do cliente e, se ele estiver ativo, assina um JWT com validade de quinze minutos. Nas requisições seguintes, um Lambda authorizer valida a assinatura na borda, com cache de trezentos segundos.

```
POST /auth { "cpf": "529.982.247-25" }  
→ 200 { accessToken, tokenType: "Bearer", expiresIn: 900 }  
→ 400 CPF_INVALIDO (sem consulta ao banco)  
→ 401 AUTENTICACAO_RECUSADA (CPF sem cadastro, cliente inativo ou bloqueado)
```

CPF sem cadastro e cadastro sem permissão recebem a mesma resposta, com status 401, cabeçalhos e corpo idênticos. O motivo específico é registrado apenas no log, de modo que o endpoint não permita descobrir quais CPFs estão cadastrados.

O subject do token é o UUID do cliente, e não o CPF, para evitar que esse dado apareça nos logs das requisições subsequentes.

Figura 4 — Sequência de autenticação por CPF e consumo de rota protegida



Fonte: elaborado pelos autores.

3.2 Escolha do gateway

O material das disciplinas apresenta três opções de gateway. A comparação entre elas é apresentada a seguir:

Opção	Pontos favoráveis	Motivo da não escolha
Azure APIM	Portal do desenvolvedor, políticas e cache prontos	Exigiria trocar de provedor e abandonar a infraestrutura da Fase 2
Kong (open source)	Independente de provedor, plugin JWT com Consumers	Seria executado dentro do cluster que deveria proteger
AWS API Gateway HTTP API	Integração nativa com Lambda,	Opção escolhida

	menor custo e latência	
--	------------------------	--

Entre os modos do AWS API Gateway, optou-se pelo HTTP API em razão do custo e da latência. O modo REST API oferece chave de API, throttling por cliente e integração com WAF, recursos que não são exigidos nesta entrega. Caso o rate limiting por cliente se torne um requisito, essa decisão deverá ser revista.

3.3 Rotas fora da autenticação por CPF

Três rotas não passam pela autenticação por CPF:

- **GET /api/ordens/{id}/status:** consulta do status de uma ordem por identificador opaco.
- **Rotas de aprovação externa:** recebem a decisão do cliente a partir do link enviado por e-mail e são autenticadas por um token de uso único com validade própria.

Essas rotas são chamadas por um sistema externo que não possui token JWT, e submetê-las ao authorizer inviabilizaria a integração. A decisão está registrada no diagrama de sequência e na especificação do gateway.

As rotas de aprovação externa são protegidas pelo token gerado junto com o orçamento:

Aspecto	Implementação
Geração	32 bytes de SecureRandom em Base64 URL-safe (256 bits)
Vínculo	armazenado na própria ordem; aceito somente na rota dessa ordem
Validade	168 horas a partir da geração do orçamento, configurável
Uso único	removido da ordem após a aprovação ou a reprovação
Erro	token divergente, expirado ou já utilizado é recusado com 401
Links do e-mail	o GET exibe uma página de confirmação e não altera a ordem; a decisão é registrada pelo POST enviado pela página
Distribuição	enviado apenas no e-mail do cliente; o access log do gateway não registra a query string
Testes	OrdemServicoTest, OrdemServicoUseCasesTest,

	ReprovarOrcamentoUseCaseTest e OrdemServicoIntegrationTest
--	---

3.4 Papéis de acesso

A Lambda emite dois papéis, ambos com autenticação por CPF, e a aplicação aplica as permissões correspondentes (ADR-014):

Papel	Rota de autenticação	Acesso na aplicação
FUNCIONARIO	POST /auth/funcionarios	todas as rotas de gestão: clientes, veículos, catálogo, estoque, ciclo da OS e relatório
CLIENTE	POST /auth	leitura das próprias ordens de serviço e dos próprios veículos; recursos de outros clientes retornam 404

O primeiro funcionário de homologação é cadastrado por uma migration, com o CPF informado por um secret do pipeline de infraestrutura, sem registro no repositório.

3.5 Uso de Lambda authorizer em vez do authorizer nativo

O JWT authorizer nativo do HTTP API é compatível com OIDC e exige emissor, audiência e um endpoint JWKS com chaves públicas. Ele não valida tokens assinados com HMAC, pois a assinatura simétrica não possui chave pública a ser publicada.

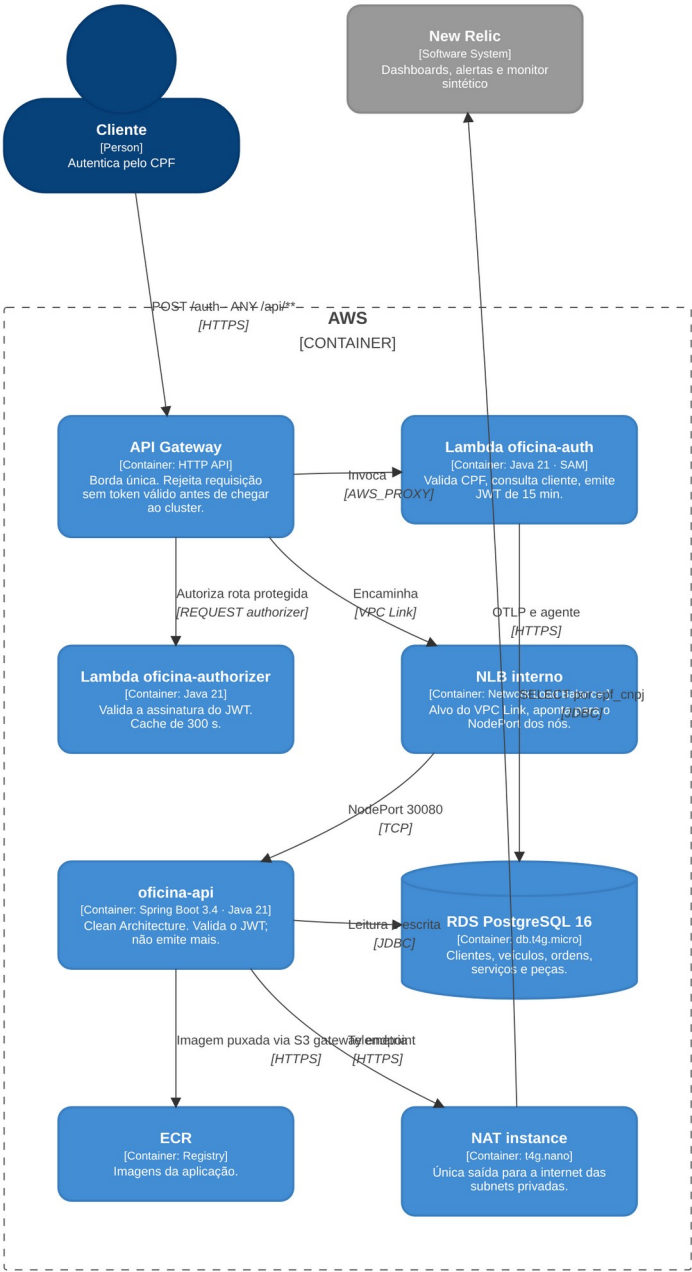
A manutenção do HMAC reduziu as alterações no JwtService existente. Como consequência, o segredo é compartilhado entre três componentes; esse ponto está registrado como débito técnico, e a migração para RS256 com JWKS é a evolução prevista.

4. Arquitetura de Nuvem

A Figura 5 apresenta o diagrama de contêineres do modelo C4, com os componentes de nuvem, as APIs, o banco de dados e o monitoramento. A fonte versionada está em docs/tech-challenge-3/diagramas/c2-containers.md. As subseções seguintes descrevem as decisões que não estão explícitas no diagrama.

Figura 5 — Diagrama de contêineres (C4, nível 2)

Sistema da Oficina — Contêineres (AWS us-east-1)



Fonte: elaborado pelos autores.

4.1 Rede privada

Na Fase 2, os nós do cluster e o banco de dados estavam em subnets públicas, protegidos por security groups. O material da disciplina de Serverless (Aula 04) aponta que um banco de dados exposto publicamente representa risco relevante, mesmo com senha forte, e que uma API em subnet pública pode ter o endereço descoberto e ser alvo de ataques.

A migração para subnets privadas exigiu definir como os recursos acessariam a internet. Foram avaliadas três opções, com preços de lista em us-east-1 e duas zonas de disponibilidade:

Opção	Cálculo	US\$/mês
NAT Gateway	US\$ 0,045/h	~33
VPC interface endpoints	US\$ 0,01/h × 4 endpoints × 2 AZs	~58
NAT instance t4g.nano + EIP	US\$ 0,0042/h + US\$ 0,005/h	~7

Os VPC interface endpoints apresentaram o maior custo, pois são cobrados por endpoint e por zona de disponibilidade. Além disso, não atenderiam à telemetria do New Relic, que é enviada para um endpoint público.

Optou-se pela NAT instance. Como efeito adicional, os nós em subnet privada deixam de utilizar IPv4 público, cobrado pela AWS desde fevereiro de 2024, o que representa uma economia de US\$ 7,30 por mês.

A NAT instance é um ponto único de falha em uma zona de disponibilidade. Em caso de indisponibilidade, o impacto é a perda de telemetria e a falha no download de imagens, sem indisponibilidade da API.

4.2 Registry de imagens no ECR

Com os nós em subnets privadas, o download de imagens do registry anterior passaria pela NAT instance, que possui banda limitada e poderia se tornar um gargalo durante o escalonamento. Com o ECR e o gateway endpoint de S3, que não tem custo, as camadas das imagens são obtidas pela rede interna da AWS, sem passar pela NAT.

A role dos nós já possuía a política AmazonEC2ContainerRegistryReadOnly desde a Fase 2, embora ela não fosse utilizada.

4.3 Caminho até a aplicação

O API Gateway acessa o cluster por meio de um VPC Link, que aponta para um Network Load Balancer interno, responsável por encaminhar o tráfego para um NodePort fixo dos nós. O balanceador é criado pelo Terraform, e não por um Service do tipo LoadBalancer, para evitar dependência de ordem de execução entre Terraform e Kubernetes.

5. Estrutura de Repositórios e CI/CD

5.1 Os quatro repositórios

O repositório único da Fase 2 foi dividido em quatro. O repositório da aplicação foi mantido com o histórico das fases anteriores, e os outros três foram criados nesta fase a partir do conteúdo transferido.

Repositório	Conteúdo	Pipeline
tech-challenge-1	oficina-api: Spring Boot, Dockerfile, testes	build → testes → Sonar (main) → imagem no ECR do ambiente → deploy no EKS
oficina-auth-lambda	SAM Java 21, duas funções	build → testes → verificação do cluster → sam deploy → ARNs no SSM
oficina-infra-k8s	Rede, EKS, ECR, gateway, New Relic	fmt → validate → plan → apply da base; cluster por disparo manual
oficina-infra-db	RDS PostgreSQL	fmt → validate → plan; apply por disparo manual

5.2 Integração entre repositórios

Para reduzir o acoplamento entre os repositórios, a troca de informações foi concentrada no SSM Parameter Store, sob o prefixo /oficina/<ambiente>/. Cada repositório publica seus outputs e consome os dos demais por meio de data sources.

Optou-se pelo SSM em vez de terraform_remote_state para que nenhum repositório precise de permissão de leitura sobre o state de outro.

5.3 Ordem de provisionamento

O API Gateway depende do ARN da Lambda para configurar as rotas, e a Lambda depende da rede criada pelo repositório de infraestrutura. A dependência foi resolvida com dois applies do cluster, controlados por variáveis:

0. oficina-infra-k8s base: ECR e parâmetros; a imagem da aplicação já é publicada
1. oficina-infra-k8s cluster, apply 1: rede, EKS e gateway sem as rotas da Lambda
2. oficina-infra-db RDS nas subnets privadas
3. oficina-auth-lambda funções e publicação dos ARNs no SSM

4. oficina-infra-k8s cluster, apply 2: rotas da Lambda e manifests da aplicação;
o pod é iniciado e o Flyway aplica V6 e V7

Os manifests da aplicação são aplicados apenas no passo 4, pois dependem do endereço do banco de dados. Até esse passo, a Lambda está implantada, mas não recebe chamadas: sua rota é criada no mesmo apply em que a migration V6 adiciona a coluna de status consultada por ela.

A sequência se aplica a cada ambiente, com os workflows disparados na branch correspondente, e a destruição segue a ordem inversa. O workflow de destruição do cluster não prossegue enquanto o banco ou a Lambda do mesmo ambiente existirem. A ordem está documentada no README de cada repositório.

5.4 Ambientes de homologação e produção

O enunciado exige deploy automático para homologação e produção. A versão inicial do planejamento previa os dois ambientes em um único cluster e em uma única instância de banco, separados por namespace e database. Essa abordagem tem custo menor, mas não isola falhas: um teste de carga em homologação, por exemplo, consumiria recursos compartilhados com produção.

Considerando que um ambiente declarado em código, com state, pipeline e proteção próprios, não gera custo enquanto não é provisionado, os dois ambientes foram definidos de forma completa e independente (ADR-012):

	Homologação	Produção
Branch	develop	main
GitHub Environment	staging, sem aprovação	prod, com revisor
Recursos	oficina-api-staging-*, VPC própria	oficina-api-prod-*, VPC própria
State do Terraform	infra-k8s/staging/..., infra-db/staging	infra-k8s/prod/..., infra-db/prod
Parâmetros no SSM	/oficina/staging/...	/oficina/prod/...
Segredos	gerados pelo Terraform	gerados pelo Terraform

Os testes e a demonstração são realizados em homologação. O ambiente de produção está preparado e protegido, com o mesmo código e os mesmos pipelines, mas não foi provisionado, de modo que o custo de operação corresponde ao de um único ambiente.

5.5 Proteção das branches e dos pipelines

- branches develop e main protegidas contra push direto, inclusive para administradores;
- merge somente por Pull Request, com uma aprovação e status checks aprovados;
- aplicação e Lambda implantadas automaticamente após o merge; na infraestrutura, a stack de menor custo (ECR e parâmetros) é aplicada no merge, enquanto cluster e banco são provisionados por disparo manual;
- jobs de deploy verificam previamente a existência do ambiente e, na ausência de cluster ou ECR, são ignorados com aviso, sem falha e sem solicitar aprovação;
- o workflow de destruição de cada repositório não prossegue fora da ordem de dependências.

6. Banco de Dados

6.1 Justificativa da escolha

Foi mantido o PostgreSQL 16 no Amazon RDS, utilizado desde a Fase 2. As alternativas foram avaliadas na RFC-002.

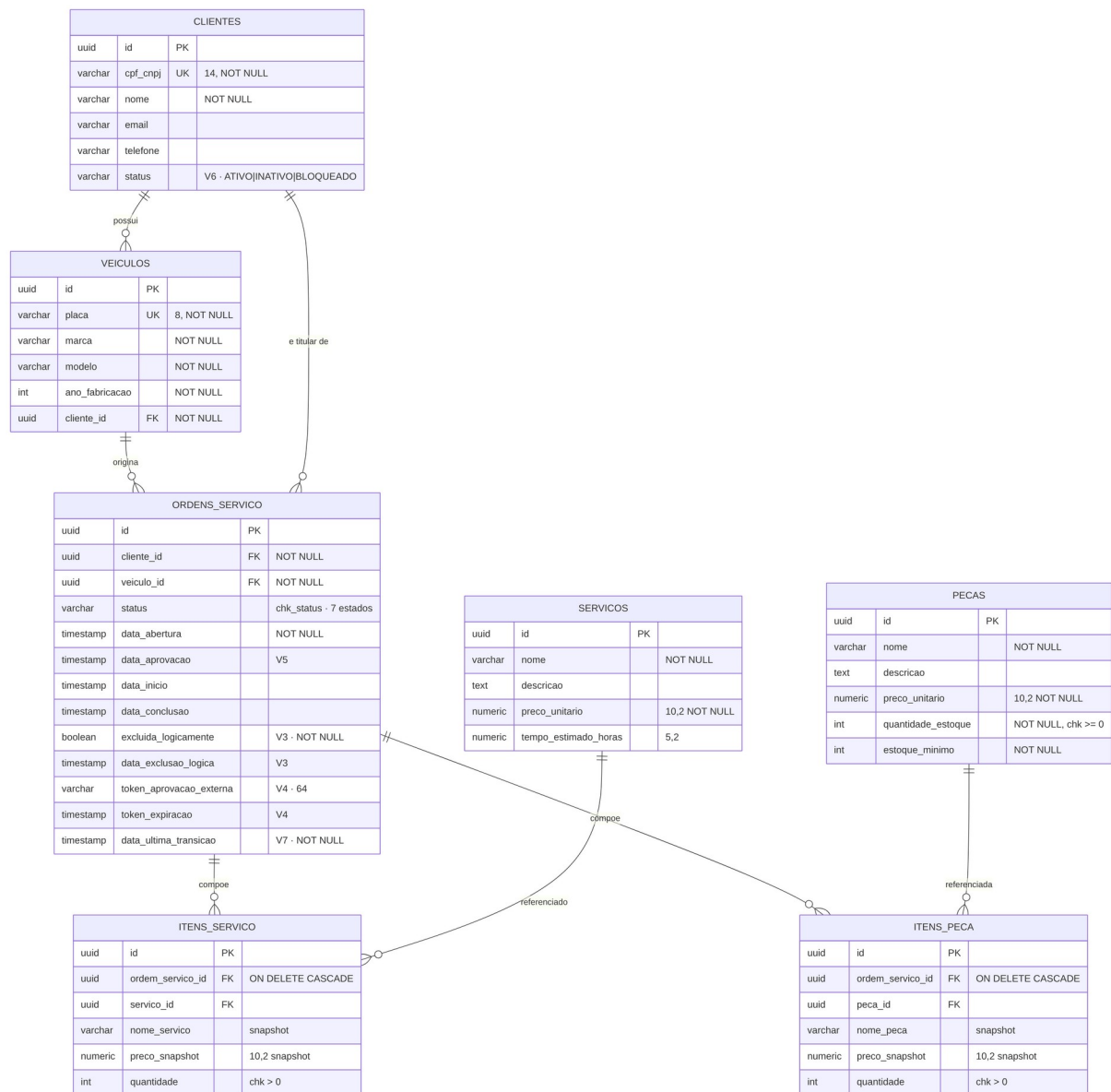
O domínio é relacional, com integridade referencial entre cinco tabelas e junções nas consultas de ordens de serviço. O schema utiliza recursos específicos do PostgreSQL: a extensão pgcrypto, a geração de UUID no servidor, o tipo numérico exato para valores monetários e constraints CHECK que representam invariantes do domínio.

Alternativa	Pontos favoráveis	Motivo da não escolha
Amazon DynamoDB	Escala melhor com Lambda, sem limite de conexões	Exigiria desnormalizar e reescrever a camada de persistência
Aurora Serverless v2	Compatível com PostgreSQL, com escala automática	A capacidade mínima custa mais que a instância utilizada
MySQL	Sem vantagem técnica para o caso	O schema depende de recursos do PostgreSQL
PostgreSQL 16 em RDS	Sem migração de dados nem reescrita, integridade preservada	Opção escolhida

6.2 Relacionamentos

A Figura 6 apresenta o diagrama entidade-relacionamento, cuja fonte versionada está em docs/tech-challenge-3/diagramas/der-modelo-relacional.md. Os relacionamentos são descritos a seguir:

Figura 6 — Diagrama entidade-relacionamento



Fonte: elaborado pelos autores.

- **Clientes 1—N Veículos:** um cliente possui zero ou mais veículos, e cada veículo pertence a um único cliente. A placa é única.
- **Ordens de Serviço:** a ordem referencia o cliente e o veículo, ainda que o cliente pudesse ser obtido a partir do veículo. A desnormalização é intencional: como a titularidade do veículo pode mudar, a ordem preserva o cliente registrado na abertura.
- **Ordem 1—N Itens:** relação de composição, na qual os itens não existem sem a ordem. O ON DELETE CASCADE reflete essa relação, que corresponde à fronteira do agregado no domínio.

- **Catálogo e Itens:** relacionamento N—N resolvido pelas tabelas de itens, que armazenam também a quantidade e o preço vigente no momento da inclusão.

O armazenamento do preço no item garante que um orçamento aprovado mantenha o valor original, mesmo que o preço do serviço ou da peça seja alterado posteriormente. A chave estrangeira para o catálogo é mantida para rastreabilidade, e o valor cobrado é obtido do item.

6.3 Invariantes protegidas no banco

As constraints atuam como segunda camada de proteção das regras de domínio, recusando escritas inválidas que não passem pela aplicação. A constraint `chk_status` restringe os sete estados da ordem; `chk_cliente_status`, as três situações cadastrais; `chk_estoque_nao_negativo` impede estoque negativo; e as restrições de unicidade de CPF e placa impedem duplicidade.

6.4 Limitações

A instância é Single-AZ, sem failover automático, e a destruição não gera snapshot final. Essas escolhas foram feitas em função do custo em ambiente de estudo, considerando que o ambiente é destruído ao fim de cada sessão de trabalho.

A classe utilizada suporta aproximadamente oitenta e cinco conexões simultâneas, o que exigiu atenção, pois a Lambda escala por invocação. Foram adotados teto de dez execuções simultâneas, imposto pelo limite de concorrência da conta e configurável como reserva no template, uma conexão por container de execução mantida em campo estático, sem pool, e timeouts curtos no driver. Caso seja necessário aumentar a concorrência, a alternativa indicada é o RDS Proxy.

7. Observabilidade

7.1 Escolha da plataforma

O enunciado permite Datadog ou New Relic, e ambos são abordados pelas disciplinas. Optou-se pelo New Relic porque seu plano gratuito é permanente, com cem gigabytes mensais de ingestão, enquanto o do Datadog é um período de avaliação de catorze dias, o que exigiria sincronizar esse período com a gravação do vídeo.

7.2 Sinais monitorados

Requisito	Implementação
Latência das APIs	Agente Java, transações do APM, percentis p95 e p99
CPU e memória do Kubernetes	Integração Kubernetes por DaemonSet, com filtro por pod
Healthchecks e uptime	Monitor sintético contra /actuator/health, em 3 localizações
Alertas de falha em ordens	Condição NRQL sobre métrica customizada de transição
Logs JSON com correlação	Structured logging do Spring Boot 3.4 (ECS), enviado pelo agente com trace.id

7.3 Logs estruturados

O Spring Boot 3.4 oferece structured logging nativo, sem necessidade de biblioteca adicional ou de configuração do logback. Foi adotado o formato ECS na saída do container, com campos padronizados de nível, logger, mensagem e contexto.

O envio ao New Relic é feito pelo agente de APM, que anexa a cada linha o trace.id e o span.id da transação em andamento e permite abrir o trace a partir do log. O Fluent Bit da integração Kubernetes permanece desativado: conforme orientação da Aula 08, com os dois mecanismos ativos cada linha seria enviada duas vezes, dobrando o consumo da cota do plano gratuito.

7.4 Dashboards de negócio

Os três painéis são alimentados por métricas customizadas emitidas pela aplicação, instrumentadas nos casos de uso e não nos controllers:

Painel exigido	Métrica	Tags
Volume diário de ordens	oficina.os.abertas	origem
Tempo médio por status	oficina.os.duracao_status	status_origem, status_destino
Erros de integração	oficina.integracao.falhas	integracao, motivo

Um dashboard técnico complementar apresenta latência por percentil, taxa de erros separada entre 4xx e 5xx, uso de recursos por pod, réplicas ativas do HPA e Apdex. Cada ambiente possui seus próprios dashboards, criados pelo mesmo código e filtrados pelo nome da aplicação e pela tag de ambiente.

7.5 Alertas e runbooks

Foram definidas seis condições de alerta por meio do Terraform, e não pelo console, conforme recomendado na Aula 10, o que evita alterações não documentadas. A nomenclatura segue o padrão apresentado na disciplina, indicando conta, ambiente, recurso, sintoma e severidade:

```
AWS-OficinaAPI-Staging-OS-FalhaTransicao-Critical
AWS-OficinaAPI-Staging-API-LatenciaP95-Warning
AWS-OficinaAPI-Staging-API-TaxaErro5xx-Critical
AWS-OficinaAPI-Staging-K8s-MemoriaPod-Warning
AWS-OficinaAPI-Staging-Integracao-Falhas-Warning
AWS-OficinaAPI-Staging-Health-Uptime-Critical
```

Cada alerta inclui o link para um runbook, conforme a orientação da Aula 05 de que um alerta deve estar associado a uma ação definida. Os seis runbooks estão em docs/runbooks e descrevem, para cada alerta, o significado, o impacto, o diagnóstico com comandos, a mitigação e o critério de escalonamento.

Nenhuma janela de avaliação é inferior a cinco minutos, para evitar disparos causados por variações momentâneas.

8. Documentação Arquitetural

A documentação está versionada em docs/tech-challenge-3, junto ao código da aplicação.

Artefato	Quantidade	Local
RFCs	3	docs/tech-challenge-3/rfcs/
ADRs	15	docs/tech-challenge-3/adrs/
Diagramas C4 (contexto, contêineres, componentes)	3	docs/tech-challenge-3/diagramas/
Diagramas de sequência	2	docs/tech-challenge-3/diagramas/
Diagrama ER com explicação dos relacionamentos	1	docs/tech-challenge-3/diagramas/
Runbooks	6	docs/runbooks/

8.1 RFC e ADR

O material da fase aborda ADR, DAS, HLD, LLD, C4 e requisitos funcionais e não funcionais, mas não trata de RFC, que é exigida pelo enunciado.

Adotou-se a seguinte convenção: a RFC é elaborada antes da decisão e registra a proposta, os comentários dos revisores e sua resolução; a ADR é elaborada após a decisão, registra a conclusão e não é alterada depois de aceita. As três RFCs tratam dos temas indicados no enunciado: escolha da nuvem, do banco de dados e da estratégia de autenticação.

8.2 Ciclo de vida das ADRs

O ciclo de vida das ADRs segue o processo apresentado na Aula 05: a ADR é criada com status Proposta, passa por revisão e, quando aceita, não é mais editada. Uma mudança de decisão é registrada em nova ADR, que marca a anterior como Superada.

A ADR-011 é retroativa e documenta o uso de HPA, implementado na Fase 2 sem registro. O enunciado cita o HPA como exemplo de decisão a ser documentada, e a elaboração da ADR identificou uma inconsistência: o HPA permitia até quatro réplicas, enquanto o node group estava limitado a três nós. A inconsistência foi corrigida nesta fase.

A mesma ADR registrou como risco a escala prematura pela métrica de memória e recomendou validá-la com teste de carga. O teste em homologação confirmou o risco: a

memória permanecia acima do alvo mesmo com a aplicação ociosa, porque a JVM retém o heap alocado, e o autoscaler não retornava ao número mínimo de réplicas. A ADR-015 registra a correção, com a escala passando a considerar apenas CPU, e supera a ADR-011 nesse ponto. O caso exemplifica o ciclo de vida descrito acima: uma decisão não é editada, e a mudança de rumo é registrada em nova ADR.

9. Qualidade de Software

Prática	Ferramenta	Execução
Testes unitários e de integração	JUnit 5, AssertJ, Mockito, Testcontainers	mvn verify
Cobertura	JaCoCo	Pipeline, relatório publicado
Análise estática	SonarCloud	Pipeline, a cada merge na main
Vulnerabilidades em dependências	OWASP Dependency-Check	Sob demanda, mvn dependency-check:check
Políticas IAM	IAM Access Analyzer e simulação de policy	Antes de aplicar cada alteração
Formatação e lint de infraestrutura	terraform fmt -check e tfint	Pipeline dos repositórios de infraestrutura
Segurança de infraestrutura	checkov, com supressões justificadas no README	Pipeline dos repositórios de infraestrutura
Contrato da API	docs/openapi.json comparado com a especificação gerada	Pipeline da aplicação
Validação de template	sam validate --lint	Pipeline da Lambda

Camada da aplicação	Classes	Métodos	Tipo
Domínio	5	34	unitário, Java puro
Aplicação	8	71	unitário, com Mockito
Infraestrutura	3	15	unitário (JWT, métricas, notificação)
Integração	1	14	Spring completo com PostgreSQL via Testcontainers

Os métodos resultam em cento e quarenta e sete casos executados, com 91% de cobertura de linhas. O build falha abaixo de 85% de linhas e 70% de branches. O teste de integração executa todas as migrations em um PostgreSQL real e compara a especificação OpenAPI gerada pela aplicação com o arquivo versionado docs/openapi.json. A Lambda possui quarenta e quatro casos de teste, incluindo consultas contra PostgreSQL via Testcontainers. A descrição de cada classe está em docs/testes.md, no repositório da aplicação, e no README da Lambda.

Em todos os repositórios, o pipeline executa os testes ou a validação a cada push, em qualquer branch: mvn verify na aplicação, mvnw verify na Lambda e terraform fmt, validate, tfint e checkov nos repositórios de infraestrutura. Esses jobs são status checks obrigatórios para o merge em develop e main.

9.1 Validação do ambiente provisionado

Os testes automatizados cobrem o código, mas não a integração entre os quatro repositórios na nuvem. Para verificá-la, o ambiente de homologação foi provisionado e destruído por completo, pelos workflows, com a demonstração executada no meio do ciclo.

Etapa	Duração	Verificação
Provisionamento	34 min	Rede, EKS, RDS, Lambda, rotas do gateway e manifests
Autenticação	—	200 para o funcionário cadastrado, 401 para CPF sem cadastro e 400 para dígito inválido
Rotas protegidas	—	401 sem token no gateway; 403 para o cliente nas rotas de gestão
Teste de carga	5 min	23.135 requisições, nenhuma falha, p95 de 323 ms
Escalabilidade	—	HPA de duas para quatro réplicas por CPU e retorno ao mínimo
Observabilidade	—	Transações no APM, métricas de pods, métricas de negócio e log com identificador de trace
Destruição	22 min	Nenhum recurso remanescente no console

O exercício encontrou defeitos que os testes automatizados não alcançavam, entre eles o envio do nome do stage no caminho pela integração do gateway, que fazia toda rota da aplicação responder 403, e a escala permanente no teto de réplicas pela métrica de memória, tratada na ADR-015. As correções estão nos repositórios correspondentes.

9.2 Validação de CPF em dois repositórios

A validação de CPF está implementada no value object do domínio da aplicação e na Lambda. A separação em repositórios impede o compartilhamento direto do código, e a publicação de um artefato comum exigiria um pipeline de publicação adicional.

Para evitar divergência entre as duas implementações, o teste da Lambda reproduz os mesmos casos-limite do teste da aplicação: CPF com e sem máscara, dígito verificador inválido, dígitos repetidos e tamanho incorreto. A duplicação está registrada como débito técnico.

10. Segurança

10.1 Medidas adotadas

- **Rede privada:** nós e banco de dados não possuem IP público; a entrada ocorre pelo API Gateway e a saída, pela NAT instance.
- **Validação na borda:** requisições sem token válido são rejeitadas no gateway, antes de alcançar o cluster.
- **Defesa em profundidade:** além do gateway, a aplicação valida o token e mantém as regras de autorização sobre os recursos.
- **Gestão de segredos:** o segredo do JWT e a senha do banco são gerados pelo Terraform, com valores distintos por ambiente, e armazenados no SSM Parameter Store como SecureString, sem passar pelo GitHub.
- **Pipelines com menor privilégio:** autenticação por OIDC, com uma role por repositório restrita aos recursos gerenciados por aquele pipeline (ADR-013). A trust policy aceita apenas as branches develop e main e seus respectivos environments.
- **Limite de permissões:** as roles criadas pelo pipeline da Lambda recebem um permissions boundary, que impede a ampliação das permissões da função pelo próprio pipeline.
- **Papéis de acesso:** rotas de gestão restritas ao papel FUNCIONARIO; clientes acessam apenas os próprios dados (ADR-014).
- **Proteção do CPF:** apenas os três últimos dígitos são registrados em log, e o subject do token é o UUID do usuário.

10.2 Autenticação por CPF: limitação conhecida

A autenticação por CPF segue o que o enunciado especifica. Ela possui, porém, uma limitação conhecida: o CPF é um identificador, e não uma credencial secreta. Em uso real, quem conhecesse o CPF de um cliente conseguiria obter um token em nome dele.

A evolução prevista é a adoção de um segundo fator, como um código de uso único enviado por e-mail, aproveitando o serviço de notificação já existente no sistema. Como a mudança do fator de autenticação não faz parte do escopo desta entrega, a melhoria está registrada como débito técnico (DT-06) e planejada para uma etapa futura. A mesma limitação vale para o funcionário, que também se autentica apenas pelo CPF.

As seguintes mitigações foram aplicadas:

Mitigação	Risco reduzido
-----------	----------------

Resposta 401 idêntica para CPF sem cadastro e cadastro inativo	Enumeração de CPFs cadastrados
Throttling de 10 req/s na rota /auth	Tentativas em volume
Token com validade de 15 minutos, sem refresh	Uso prolongado de um token vazado
CPF fora dos logs e do subject do token	Exposição do CPF por telemetria

11. Débitos Técnicos da Fase 3

O registro completo, com situação atual, causa raiz, melhoria planejada, justificativa de escopo e impacto de cada item, está em docs/tech-challenge-3/debitos-tecnicos.md. A tabela a seguir resume os débitos:

ID	Débito	Prioridade	Risco
DT-01	E-mail de orçamento sem detalhamento de itens (herdado da Fase 2)	Baixa	Baixo
DT-02	Segredo JWT simétrico compartilhado entre três componentes	Média	Médio
DT-03	Revogação de token não imediata (janela de 15 minutos)	Média	Médio
DT-04	NAT instance como ponto único de falha	Média	Baixo
DT-05	Homologação e produção compartilhavam cluster e banco (quitado, ADR-012)	—	—
DT-06	Autenticação por CPF sem segundo fator	Alta	Alto
DT-07	Tabela refresh_tokens sem uso no schema	Baixa	Nenhum
DT-08	Validação de CPF duplicada entre repositórios	Média	Médio
DT-09	Ambientes na mesma conta AWS, com role de pipeline compartilhada	Média	Médio
DT-10	Produção preparada e não provisionada	Baixa	Baixo

Cada item foi avaliado quanto ao custo da correção e ao benefício esperado, e está registrado com a respectiva melhoria planejada.

12. Conclusão

A Fase 3 teve como objetivo preparar o sistema para operação em ambiente corporativo. Não foram adicionadas regras de negócio; as alterações concentraram-se na segurança, na operação e na visibilidade do sistema.

A emissão de tokens foi transferida para uma função serverless protegida por um API Gateway, o que centralizou a identidade em um único emissor. Os nós e o banco de dados passaram a operar em subnets privadas, e a forma de acesso à internet foi definida a partir da comparação de custos entre as alternativas. O repositório único foi dividido em quatro, com pipelines próprios e integração pelo SSM Parameter Store, e os ambientes de homologação e produção foram segregados sem aumento do custo de operação.

No aspecto de observabilidade, o sistema passou a contar com métricas de negócio, logs estruturados correlacionados aos traces e alertas associados a runbooks.

A Clean Architecture adotada desde a Fase 1 permitiu que a troca do emissor do token e a inclusão das métricas de negócio fossem realizadas sem alterações nas regras de domínio, com as mudanças concentradas na camada de infraestrutura.

A fase foi encerrada com nove débitos técnicos abertos e um quitado. Cada um está registrado com a melhoria planejada e a justificativa do adiamento. O principal deles, a autenticação apenas por CPF, atende ao que o enunciado especifica, conta com mitigações e tem a adoção de um segundo fator prevista para uma etapa futura.

13. Referências Bibliográficas

AMAZON WEB SERVICES. ADR Process — Prescriptive Guidance. 2022. Disponível em: <https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/>. Acesso em: 4 set. 2026.

AMAZON WEB SERVICES. Criação de uma API HTTP com uma integração privada a um serviço do Amazon ECS. Disponível em: https://docs.aws.amazon.com/pt_br/apigateway/latest/developerguide/http-api-private-integration.html. Acesso em: 4 set. 2026.

AMAZON WEB SERVICES. What is AWS Lambda? Disponível em: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>. Acesso em: 4 set. 2026.

BASS, L.; CLEMENTS, P.; KAZMAN, R. Software Architecture in Practice. 3. ed. Boston: Addison-Wesley Professional, 2012.

BROWN, S. The C4 model for visualizing software architecture. Disponível em: <https://c4model.com/>. Acesso em: 4 set. 2026.

GOOGLE. Site Reliability Engineering. Disponível em: <https://sre.google/>. Acesso em: 4 set. 2026.

MARTIN, R. C. Arquitetura Limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

NEW RELIC. Apdex: meça a satisfação do usuário. Disponível em: <https://docs.newrelic.com/pt/docs/apm/new-relic-apm/apdex/apdex-measure-user-satisfaction/>. Acesso em: 4 set. 2026.

NEW RELIC. Logs contextualizados APM. Disponível em: <https://docs.newrelic.com/pt/docs/logs/logs-context/get-started-logs-context/>. Acesso em: 4 set. 2026.

NEW RELIC. NRQL reference. Disponível em: <https://docs.newrelic.com/docs/query-your-data/nrql-new-relic-query-language/>. Acesso em: 4 set. 2026.

NYGARD, M. Documenting Architecture Decisions. 2011. Disponível em: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 4 set. 2026.

RICHARDS, M.; FORD, N. Fundamentos da Arquitetura de Software: uma abordagem para engenharia de software. Porto Alegre: Bookman, 2020.

VMWARE. Spring Boot Reference — Structured Logging. Disponível em: <https://docs.spring.io/spring-boot/reference/features/logging.html>. Acesso em: 4 set. 2026.

FIAP. Material das disciplinas da Fase 3 — Monitoramento e Acesso; Monitoramento e Acesso Avançado; API Gateway; Desenvolvimento Serverless; Documentação de Arquitetura de Soluções. PosTech Arquitetura de Software, 2026.